StudyForge Stage 2 Implementation Report

Course: EECS3311 Software Design, Fall 2026

Repository: ZaaidAhmed/StudyForge

Report evidence checked: November 7, 2026

Status: Java 21 tests, package build, fresh-JAR web smoke, CLI command discovery, and public link response checks passed locally. Exact in-video segment review, provider billing exports, final GitHub publication, and instructor-access confirmation remain evidence gates.

1. Project overview and implementation status

StudyForge is a local-first learning workspace for students. It imports course PDFs, text, and Markdown, stores normalized source chunks, provides explainable lexical retrieval, and supports grounded learning activities. A Spring MVC/Thymeleaf GUI and Picocli CLI use the shared `StudyAgentFacade` and application services. Spring Data JPA and file-backed H2 store course and learning data. The provider-neutral `LlmClient` routes to local Ollama or hosted Gemini. The agent performs intent analysis, bounded planning, allowlisted course-scoped tool execution, recent-history retrieval, and response synthesis; model proposals are validated before tools run.

The Stage 1 proposal contains F01-F14. All fourteen are implemented in the current codebase, with differences documented in Section 4 and the crosswalk. F15, runtime selection of local/cloud AI providers, is a Stage 2 addition. The codebase has working GUI and CLI paths for the commands listed in the README; F09 quiz taking/grading and F10 wrong-answer explanations are intentionally GUI-only. This report distinguishes code support from feature-specific public video evidence.

Known limitations include no OCR for scanned/image-only PDFs, no semantic/vector retrieval, local single-user operation without authentication, and provider-dependent generation quality/latency. These are outside the proposal's initial scope. Stage 2 requires basic verification; the full deterministic test plan and KUMA behavioral evaluation are Stage 3 work.

2. Feature-to-implementation mapping

Every Stage 1 proposal feature is retained in the mapping. “GUI/CLI” describes implemented interfaces; where an interface is intentionally absent, it is stated. Stage 1 use-case and sequence identifiers refer to the proposal's UC01-UC14 and SD01-SD12 diagrams. The feature-specific evidence index is in Section 7 and `demo-index.md`.

Feature	Stage 1 reference; status/type	Implemented classes and responsibilities; key operations	Patterns	Verification and demo
F01 Course workspace	UC01, SD11; implemented; deterministic	`CourseController` handles GUI requests; `CourseService` validates and manages courses (`createCourse`, `getAllCourses`, `getCourseByCode`, `updateCourse`, `deleteCourse`); `CourseRepository` persists; Picocli course commands expose supported create/list/show operations.	MVC, Facade, Command, Repository	`CourseServiceTest`, `CourseRepositoryTest`, `CourseControllerTest`, `CliCommandTreeTest`; demo F01.
F02 Material import and processing	UC02, SD04/SD11; implemented; deterministic	`MaterialController` and `MaterialAddCommand` accept files; `CourseMaterialService.importMaterial` validates course/type/size/hash, calls storage/extractor/normalizer/chunker collaborators, and persists `CourseDocument`/`DocumentChunk`.	MVC, Strategy-like extractor polymorphism, Facade, Command, Repository	`CourseMaterialServiceTest`, `DocumentProcessingTest`, `MaterialControllerTest`; demo F02.
F03 Course material retrieval	UC03, SD11; implemented; deterministic	`CourseSearchService.search` scopes READY chunks to the course; `QueryNormalizer` and `LexicalRetrievalScorer` rank them; `MaterialController`/`MaterialSearchCommand` present source-aware results.	MVC, Facade, Command, Repository	`CourseSearchServiceTest`, `LexicalRetrievalScorerTest`, `MaterialSearchCommandTest`; demo F03.
F04 Grounded course Q&A	UC04, SD01; implemented; hybrid	`GroundedQuestionService.answer` retrieves through `CourseSearchService`, creates bounded evidence with `GroundingContextBuilder`, builds a prompt, calls `LlmClient`, parses structured output, and validates source IDs; `AskController`/`AskCommand` expose it.	MVC, Facade, Adapter, Command	`GroundedQuestionServiceTest`, `AskControllerTest`; demo F04.
F05 Document/topic summaries	UC05, SD05; implemented; hybrid	`SummarizationService.summarizeDocument` and `summarizeTopic` validate scope, assemble bounded evidence, call the provider, parse and validate cited sources; `LearningController`/`SummaryCommand` present results.	MVC, Facade, Adapter, Command	`SummarizationServiceTest`; demo F05.
F06 Key concept extraction	UC06, SD06; implemented; hybrid	`ConceptExtractionService.extract` requests structured concepts, validates source IDs, and deterministically merges duplicates; `LearningController`/`ConceptsCommand` expose extraction.	MVC, Facade, Adapter, Command	`ConceptExtractionServiceTest`, structured-parser coverage; demo F06.
F07 Flashcard generation/review	UC07, SD07; implemented; hybrid generation and deterministic review	`FlashcardGenerationService.generate` validates document scope, extracts concepts, generates/retries/backfills, validates/deduplicates and persists cards/sources; `FlashcardReviewService.review` validates ownership/rating and appends review records; web and CLI commands expose supported operations.	MVC, Facade, Adapter, Command, Repository	`FlashcardGenerationServiceTest`, `FlashcardReviewServiceTest`, `FlashcardControllerTest`; demo F07.
F08 Grounded quiz generation	UC08, SD08; implemented; hybrid	`QuizGenerationService.generate` builds document-grounded questions, parses and validates type/options/sources, rejects duplicates, recovers where possible, and persists the quiz; `QuizController`/`QuizCommand` provide GUI and CLI generation/listing.	MVC, Facade, Adapter, Command, Repository	`QuizGenerationServiceTest`, `QuizControllerTest`, `CliCommandTreeTest`; demo F08.
F09 Quiz taking/grading	UC09, SD09; implemented; hybrid for legacy short answers	`QuizAttemptService.submit` validates quiz/question ownership, grades objective items deterministically, uses source-checked model grading for legacy short answers with deterministic fallback, and persists `QuizAttempt`/`QuizResponse`; GUI only.	MVC, Facade, Repository	`QuizAttemptServiceTest`, quiz MVC tests; demo F09.

Feature	Stage 1 reference; status/type	Implemented classes and responsibilities; key operations	Patterns	Verification and demo
F10 Evidence-based wrong-answer explanations	UC10, SD10; implemented; hybrid	`WrongAnswerExplanationService.explain` validates course/quiz/question/attempt, uses the persisted answer and question sources, validates the response/citations, and leaves the grade unchanged; GUI only.	MVC, Facade, Adapter, Repository	`WrongAnswerExplanationServiceTest`, quiz MVC tests; demo F10.
F11 Progress/weak-topic analysis	UC11, SD12; implemented; deterministic	`ProgressService.summarize` aggregates persisted flashcard ratings and quiz attempts into overall and topic scores/status; progress controller, agent tools, and `ProgressCommand` present it.	MVC, Facade, Command, Repository	`ProgressServiceTest`, `ProgressController`/CLI smoke; demo F11.
F12 Personalized study plans	UC12, SD03; implemented; deterministic	`StudyPlanService.create` validates dates/budget, selects `StudyPlanningStrategy`, enforces allocation constraints, persists plan/items; balanced, weak-topic-focused, and exam-cram strategies implement distinct allocations; GUI/CLI supported.	MVC, Facade, Strategy, Command, Repository	`StudyPlanServiceTest`, `StudyPlanningStrategyTest`, study-plan MVC tests; demo F12.
F13 Recommendations and bounded agent assistance	UC13, SD02; implemented; hybrid	`RecommendationService.recommend` selects a deterministic next action; `AgentCoordinator` uses `IntentAnalyzer`, `TaskPlanner`, `AgentPlanValidator`, `AgentToolManager`, `MemoryManager`, and `ResponseSynthesizer` for bounded, course-scoped tasks; `RecommendationCommand`/`AgentCommand` expose CLI paths.	MVC, Facade, Adapter, Command	`RecommendationServiceTest`, `AgentPlanningTest`, `AgentToolManagerTest`, `AgentBehavioralTest`, agent MVC tests; demo F13.
F14 Learning history/bounded memory	UC14, SD12; implemented; deterministic	`LearningHistoryService` derives newest-first events from persisted flashcard reviews, quiz attempts, and study plans, capped at 100; `MemoryManager.recent` supplies ten recent events to synthesis; `HistoryController`/`HistoryCommand` expose history.	MVC, Facade, Command, Repository	`LearningHistoryServiceTest`, history MVC tests; demo F14.
F15 Local/cloud AI provider selection	New in Stage 2; implemented; deterministic configuration/routing	`AiStatusController` validates GUI provider/model selection; `AiProperties` stores active runtime configuration; `RoutingLlmClient` delegates to `OllamaLlmAdapter` or `GeminiLlmAdapter`; CLI selects via environment and `AiStatusCommand` reports the active provider/model. No dedicated CLI switch command exists.	MVC, Adapter, Facade, Command	`GeminiLlmAdapterTest`, `OllamaLlmAdapterTest`, configuration/routing coverage; demo F15.

The table records representative tests rather than claiming that each listed class has exhaustive coverage. The final clean test result is in Section 6.

3. Implemented design patterns

The following five patterns are distinct and visible in code. Repository is also used as a persistence pattern, but is not needed to meet the five-pattern minimum.

Model-View-Controller (MVC)

Problem: Separate HTTP handling and HTML rendering from reusable course behavior. **Participants/roles:** Spring MVC controllers parse requests and choose views; application services execute use cases; Thymeleaf templates render state. **Evidence:** `web/Controller.java`, `application/service/`, and `src/main/resources/templates/`. **Stage 1:** Matches the proposal. It allows the GUI and CLI to use the same service rules.

Facade

Problem: Give the GUI, CLI, and bounded agent a stable application boundary instead of exposing collaborator graphs. **Participants/roles:** `StudyAgentFacade` delegates use-case calls to application services and `AgentCoordinator`; MVC controllers and Picocli command objects are clients. **Evidence:** `StudyAgentFacade` operations including `askQuestion`, generation/listing, grading, progress, and `agentAssist`. **Stage 1:** Matches the proposed shared facade.

Adapter

Problem: Keep provider HTTP protocols out of feature services. **Participants/roles:** `LlmClient` is the provider-neutral port; `OllamaLlmAdapter` and `GeminiLlmAdapter` translate provider requests/responses; `RoutingLlmClient` selects the configured adapter. **Evidence:** `application/ai/LlmClient`, `infrastructure/ai/*LlmAdapter`, and `RoutingLlmClient`. **Stage 1:** Preserves the planned Ollama Adapter and extends it with Gemini and runtime routing as F15.

Strategy

Problem: Vary study allocation without embedding every policy in one scheduler. **Participants/roles:** `StudyPlanningStrategy` defines the policy contract; `BalancedStudyStrategy`, `WeakTopicFocusedStrategy`, and `ExamCramStrategy` produce activities; `StudyPlanService` validates constraints and persists. **Evidence:** `StudyPlanService.create` and strategy `build` methods. **Stage 1:** Matches the proposal's strategy-based scheduling.

Command

Problem: Represent supported CLI actions as independently parsed operations and keep argument handling out of services. **Participants/roles:** Picocli is the invoker; concrete command classes such as `CourseCreateCommand`, `MaterialSearchCommand`, `QuizCommand.Generate`, `RecommendationCommand`, and `AgentCommand` hold arguments and invoke `StudyAgentFacade`/services. **Evidence:** `cli/*Command.java` and each command's `run`. **Stage 1:** Matches the proposed Picocli Command pattern.

Supplementary Repository pattern: Spring Data repositories isolate persistence for course, documents/chunks, flashcards/reviews, quizzes/attempts, and study plans. This supports testability and scoped queries; it is documented as supplementary because the five patterns above already satisfy the required count.

4. Changes from the Stage 1 design and rationale

Original design	Implemented change	Reason	Impact
F01 course operations and shared facade	Separate CLI Spring entry point/profile; course lookup uses `getCourseByCode` and DTO boundaries.	Avoid starting the web server for CLI use and clarify the service contract.	Same course behavior and shared application logic; no feature removed.
F02 `MaterialImportService`/`FileStorage` names	`CourseMaterialService`/`FileStorageService`; extractor implementations selected by type.	Align names with the existing service convention and isolate format-specific extraction.	Import validation, safe storage, extraction, normalization, chunking, and persistence remain traceable to UC02.
F03 `MaterialSearchService`	`CourseSearchService` with deterministic lexical ranking behind `RetrievalScorer`.	Clarify course scope and keep retrieval explainable; embeddings/vector search were excluded in Stage 1.	Search remains deterministic and source-addressable; no semantic search is claimed.
Stage 1 proposed local Ollama `qwen2.5-coder:14b` provider	Provider-neutral `LlmClient` routes to Ollama `qwen3:4b` or Gemini; CLI selection uses process environment; F15 GUI selection is new.	Fit local hardware and allow a hosted option while preserving provider isolation.	Adds F15; API credentials remain local configuration. Exact deployment/configuration behavior is described in README.
Quiz and flashcard generation used bounded single/limited retry contracts	Generation now includes structured parsing, quality/source validation, semantic-ish deduplication, retry/backfill, and partial-result reporting.	Observed provider schema variance, noisy source text, and duplicate/short generation outputs.	Safer persisted content and clearer shortfall reporting; it cannot guarantee a target count when unique grounded content is exhausted.
Stage 1 browser document view	Application-controlled page rendering for document chat.	Keep document Q&A inside the StudyForge workspace and control navigation.	Preserves selected-document Q&A; no browser PDF embedding is required.
Quiz deletion without explicit dependent cleanup	Course-scoped deletion removes dependent attempts before the quiz.	Prevent orphaned attempt rows.	Deletion remains scoped and consistent with persistence constraints.
Stage 1 F14 bounded recent memory	History page groups/filter activities; memory remains ten recent events and is passed selectively to relevant agent synthesis.	Make persisted activity easier to inspect while keeping memory bounded and relevant.	History remains derived only from flashcard reviews, quiz attempts, and plans; no transcripts or hidden reasoning are stored.

The Stage 1 proposal remains unmodified as the baseline. Updated implementation UML and crosswalks are in `docs/stage2/uml/` and `design-to-implementation-traceability.md`.

5. AI tools and human collaboration

The application uses local Ollama and optionally hosted Gemini for student-facing AI features. Development also used the Codex coding assistant for architecture discussion, implementation, debugging, tests, UX iteration, and documentation. The retained phase log identifies GPT-5 for Phase 1-3; later entries in the retained project log do not record a precise Codex model or usage ledger. No model names or token totals are inferred beyond those records.

Tool/platform	Model evidence	Representative task and contribution	Human role/outcome	Usage/cost record
Codex coding assistant	GPT-5 recorded for Phases 1-3; exact model for later sessions is not present in retained project records.	Implemented/refined shared services, grounded-generation validation, quiz/flashcard recovery, agent workflows, history, study-plan guidance, README, and release documentation.	The student specified scope and acceptance behavior, reviewed the implementation, supplied screenshots/errors, and directed corrections and verification.	Aggregate development token and monetary totals are not exposed in the retained repository log. Do not treat as zero or estimate. Obtain from the coding platform's usage/billing export if available; otherwise report the platform's limitation.
Ollama runtime	`qwen3:4b`; `qwen-claude-14b:latest` also installed/used in documented attempts.	Local provider for grounded learning and content generation; documented provider attempts sometimes returned controlled structured-output failures.	Application validates model output; failed/invalid output is not silently persisted.	No hosted API charge; local compute cost is not measured by the app.
Gemini API	Configurable model is listed in README; the project log does not establish a complete usage total.	Optional hosted provider for app AI requests when configured.	User supplies and keeps the API key local; no secret is included in the repository/report.	Per-account monetary/token total must come from provider usage records; not recoverable from source code or local build results.

Representative interactions are recorded in `ai-collaboration-log.md`. That log distinguishes retained evidence from missing exact model/usage telemetry. Before submission, replace any unresolved usage record with an exported factual total or an explicit platform limitation, not an estimate.

6. Development and verification summary

Verification performed for this report

- **Date/environment:** November 7, 2026, OpenJDK 21.0.12.1, Maven Daemon 1.0.6.
- **Automated suite:** `mvnd clean test` completed successfully: **207 tests, 0 failures, 0 errors, 0 skipped**.
- **Package:** `mvnd package` completed successfully on Java 21 on November 7, 2026; it reran the suite with **207 tests, 0 failures, 0 errors, 0 skipped** and produced `target/studyforge-ai-0.1.0-SNAPSHOT.jar`.
- **Fresh-JAR web smoke:** `scripts/verify.ps1 -SkipBuild -Port 18080` started the packaged JAR and received HTTP 200 from `/`, `/courses`, and `/ai/status`.
- **CLI checks:** packaged-JAR help commands for `quiz`, `recommend`, and `agent` completed. Read-only `quiz list --course EECS3221`, `recommend --course EECS3221`, and `history --course EECS3221` also completed. This confirms command registration and these read workflows, not a full AI-backed quiz generation or agent task run.
- **Provider and live AI generation:** the test suite uses controlled clients for repeatability; a passing test suite does not establish that every external provider or generated answer is available/valid at submission time.
- **Browser and CLI demonstration:** use the public video index and README for feature invocations. F09/F10 are GUI-only; F15 CLI behavior is environment selection followed by `ai status`.

The tests cover deterministic services, parsing, retrieval, repository behavior, controllers/templates, CLI command registration, and AI adapters. Stage 2 requires basic verification; Stage 3 calls for broader deterministic coverage and KUMA behavioral evaluation. No KUMA result is claimed here.

Repository and release evidence

The weekly summary lists five meaningful code commits in each reporting period (October 20-26, October 27-November 2, and November 3-9, Toronto local dates). Unauthenticated HTTP HEAD checks on November 7 returned status 200 for all 15 commit links. `.gitignore` excludes `.env`, local data/uploads, logs, build output, and local config; `.env.example` contains placeholders, not a credential. The final Git index must still be checked after staging to ensure none of those excluded classes were force-added.

7. Demonstration index

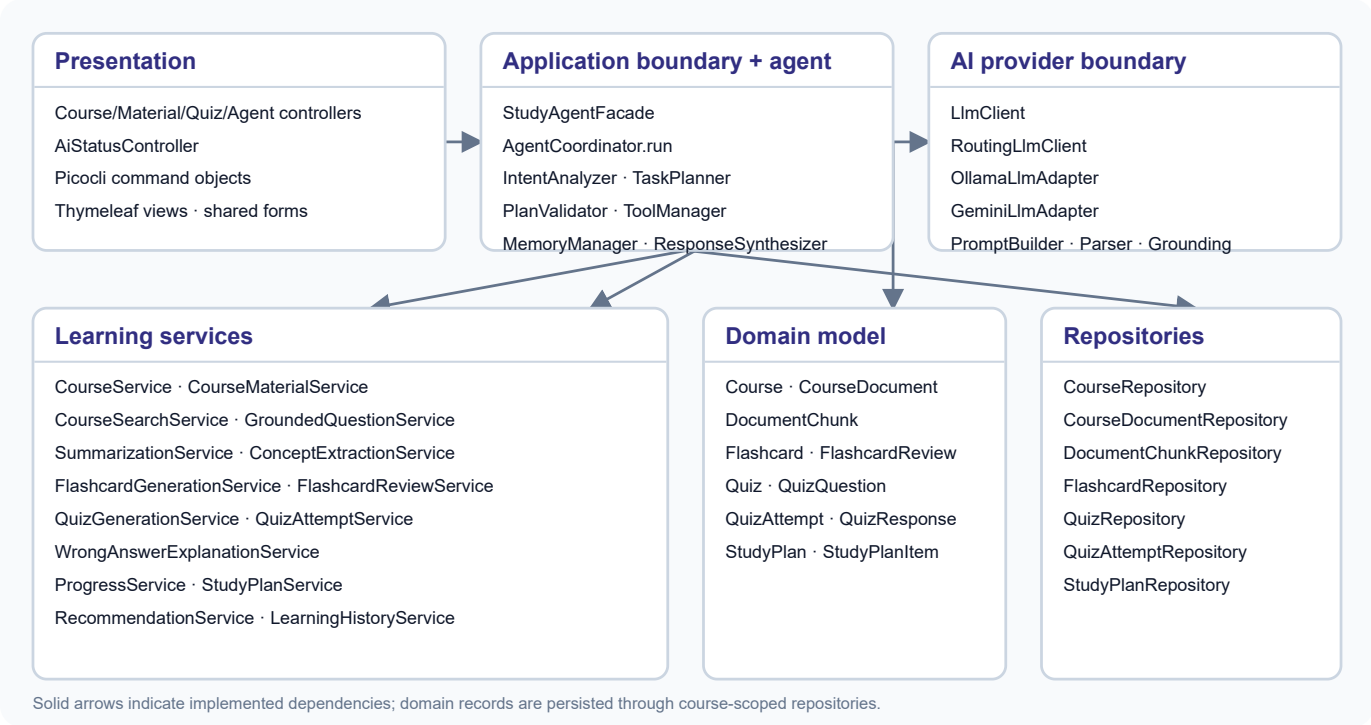
The public index is `docs/stage2/demo-index.md`. Each listed link is a dedicated feature video and points to `00:00`, the start of that standalone feature recording. Unauthenticated HTTP HEAD checks on November 7, 2026 returned status 200 for all 15 YouTube URLs and all 14 listed GitHub release assets. This verifies URL response, not that a human viewer can play every video or that every expected behavior appears in it; final playback/content review is still required.

The index identifies F08 and F13 CLI video evidence as pending even though their commands are implemented. Packaged-JAR help was checked for `quiz`, `recommend`, and `agent`; full `quiz generate` and agent task executions were not run as part of the read-only smoke. Record CLI demonstrations for `quiz generate/list`, `recommend`, and `agent`, or clearly submit the current feature clips as GUI evidence and disclose the missing CLI recording. F09/F10 are

correctly marked GUI-only. The set of recordings must collectively show both interfaces and explain each feature and relevant alternatives/errors.

8. Updated UML class diagram

The following diagram summarizes the implemented classes and important dependencies. The editable PlantUML source is `docs/stage2/uml/class-diagram.puml`.



Implemented classes and dependencies (overview).

9. Submission gates and known evidence gaps

- Review playback and exact content for all 15 YouTube videos; unauthenticated link response checks returned HTTP 200 for all YouTube and listed release asset URLs.
- Confirm exact timestamps and contents for every demonstration; create/attach F08 and F13 CLI evidence if not present in the hosted clips.
- Complete AI development-tool/model/token/money-cost records from authoritative usage data or state platform telemetry limitations precisely.
- Repeat verification on the exact commit that will be submitted; the current local package, web smoke, and CLI help checks passed, while full GUI/CLI feature workflow recording remains outstanding.
- Publish the report, diagrams, README, safe sample configuration, source, weekly links, and videos in the public repository, then verify the submitted revision and instructor access.